

MODULE 4

Addressing Modes

- The method by which the operand is specified in an instruction is called addressing modes.
- 8051 addressing modes are classified as follows.
 - Immediate addressing.
 - Register addressing.
 - Direct addressing.
 - Register Indirect addressing.
 - Indexed addressing

1. Immediate addressing modes

In this addressing mode the data is provided as a part of instruction itself.

Example

1. Mov A, #0AFH

In these instructions, the # symbol is used for immediate data

In this instruction, the immediate data is AFH, but one 0 is added at the beginning. So when the data is starting with A to F, the data should be preceded by 0.

2. Mov A, #30H

It means that immediate data 30H provided in instruction is moved into Accumulator

2. Register addressing

In register addressing mode, the instruction will specify the name of the register in which the data is available.

Example

1. Mov A, R2

Content of R2 register is copied into Accumulator

2. ADD A, R6

Content of R0 register is added with Accumulator.

- R0 – R7 will be selected from the current selection of register bank. The default register bank will be bank 0.

3. Direct Addressing mode

In direct addressing mode, the address of the data is directly specified in the instruction.

The direct address can be the address of an internal data RAM location (00H to 7FH) or the address of a special function register (80H to FFH)

Example

MOV A, 07H

The address of the R7 register of bank is 07. This instruction will move the content of the R7 register to the Accumulator.

4. Indirect or Register Indirect addressing mode

- In this mode, the instruction specifies the name of the register in which the address of the data is available
- The Internal data RAM locations (00H to 7FH) can be addressed indirectly through registers R1 and R0. The external RAM can be addressed indirectly through DPTR.

Example

MOV A, @R0

- In the instructions, the @ symbol is used for register indirect addressing.
- The internal RAM location R0 holds the address of the data. The content of the RAM location addressed by R0 is moved to the Accumulator

5. Indexed addressing mode

- Only program memory can be accessed in the index addressing. Either the DPTR or PC can be used as an index register. The destination operand is always the register A.

Example

MOV A, @ A+DPTR

- This instruction copies the contents of memory locations pointed by sum of the accumulator A and the register DPTR into the accumulator A.

Instruction set types

- The 8051 instructions can be divided into the following six groups:
 1. Data transfer instructions
 2. Arithmetic instructions
 3. Logical instructions
 4. Branch instructions
 5. Subroutine instructions

6. Bit manipulation instructions

Data transfer Instruction set

- Data transfer instructions are used to transfer data between an internal RAM location and SFR location without going through the accumulator.
- Data can also be transferred between the internal and external RAM by using indirect addressing.

1. Mov A, Rn
2. MOV A, 30H
3. Mov A, @R1
4. Mov A, # 30H
5. Mov R2, A
6. Mov R1, 40H
7. Mov R2, #20H
8. Mov 20H, A
9. Mov 30H, R2
10. Mov 20H, 40H
11. Mov 20H, @R3
12. Mov 30H, #12H
13. Mov @R1, A
14. Mov @R2, 30H
15. Mov @R2, #30H
16. MOV DPTR, #data 16

1. Mov A, Rn

Example: Mov A, R0

This instruction copies the contents of the register R0 of selected register bank to the accumulator.

2. Mov A, direct

Example: MOV A, 30H

This instruction copies the contents of memory location whose address is 30H to the accumulator.

3. Mov A, @Ri

eg. Mov A, @R1

This instruction copies the contents of memory location whose address is specified in the register R1 from selected register bank to accumulator

(or) copy the contents of the address in R1 to accumulator.

4. Mov A, #data

eg. Mov A, # 30H

This instruction copies data given within instruction (30H) into the accumulator

5. Mov Rn, A

eg. Mov R2, A

This instruction copies the contents of accumulator in R2 register of selected register bank.

6. Mov Rn, direct

eg. Mov R1, 40H

This instruction copies the contents at memory address 40H into the R1 register of the selected register bank.

7. Mov Rn, #data

eg. Mov R2, #20H

This instruction loads 20H in the registers R2 of selected register bank

8. Mov direct, A

eg. Mov 20H, A

This instruction copies the contents of accumulator to the direct memory address specified in the instruction (20H).

9 Mov direct, Rn

eg. Mov 30H, R2

This instruction copies the contents of register R2 of selected register bank to the direct memory address specified in the instruction(30H)

10. Mov direct, direct

eg. Mov 20H, 40H

This instruction copies the contents of memory location whose address is 40H to the memory location whose address is 20H

11. Mov direct, @Ri

eg. Mov 20H, @R3

This instruction copies the contents of memory location whose address is given by register R3 of selected register bank into the memory location whose address is 20H.

12. Mov direct, #data

eg. Mov 30H, #12H

This instruction copies data given within instruction (12H) into the memory location whose address is 30H

13. Mov @Ri, A

eg. Mov @R1, A

This instruction copies the contents of accumulator to the memory location whose address is given by register R1 of selected register bank.

14. Mov @Ri, direct

eg. Mov @R2, 30H

This instruction copies the contents of memory location whose address is given within the instruction (30H) into the memory location whose address is specified by register R2 of selected register bank.

Instructions to access External Data Memory.

- The MOVX instruction transfer data between the Accumulator and a byte of external data memory, hence the “X” appended to MOV.

1. MOVX, A, @Ri

This instruction copies the contents of external memory whose address is given by register into the accumulator.

2. MOVX A, @DPTR

This instruction copies the contents of external memory whose address is given by DPTR register into the accumulator.

3. MOVX @Ri, A

This instruction copies the contents of accumulator into the external memory whose address is given by the register.

4. MOVX @DPTR, A

This instruction copies the contents of accumulator into the external memory whose address is given by the DPTR register.

Instruction to Access External ROM/ Program memory

1. MOVC A, @A +DPTR

This instruction loads the accumulator from the contents of program memory whose address is given by the sum of the contents of accumulator and contents of DPTR register.

2. MOVC A, @A+PC

Copy the contents of the external ROM address formed by adding A and the PC, to A.

Data transfer with stack (PUSH and POP) Instructions

1. PUSH B

This instruction increments the stack pointer by one and stores the contents of register B to the internal RAM location addressed by the stack pointer. Otherwise no flags are affected.

2. POP ACC

This instruction copies the contents of the internal RAM location addressed by the stack pointer to the accumulator. Then the stack pointer is decremented by one.

Data Exchange instructions

1. XCH A, Rn

Eg. XCH A, R0

This instruction exchanges contents of accumulator with the contents of register R0 of selected register bank.

2. XCH A, direct

Eg. XCH A, 20H

This instruction exchanges contents of accumulator with the contents of memory whose address is given within the instruction (20H)

3. XCH A, @Ri

Eg. XCH A, @R2

This instruction exchanges the contents of accumulator with the contents of memory location whose address is given by the contents of register R2 of selected register bank.

4. XCH A, < byte >

Exchange accumulator with byte variable

5. XCHD A, @Ri

Exchange lower nibble between A and address in Ri.

Arithmetic Operations

- The arithmetic operations of 8051 include increment, decrement, addition, subtraction, multiplication, division and decimal operations.

Incrementing and Decrementing

1. INC A - Increments the accumulator by 1
2. INC Rn

Eg. INC R2 - Increments the contents of R2 by 1

3. INC direct

Eg. INC 20H – Increments contents of memory location whose address is given within the instruction(20H)

4. INC @Ri

Eg. INC @R2

Increments contents of memory location whose address is given by register R2 by 1

5. DEC A - Decrements the accumulator by 1
6. DEC Rn

Eg. DEC R2 - Decrements the contents of R2 by 1

7. DEC Direct

Eg. DEC 20H – Decrements the contents of memory location whose address is 20H by 1

8. DEC @Rn

Eg. DEC @R2

Decrements the contents of memory location whose address is given by register R2 by 1

Addition

1. ADD A, Rn

Eg. ADD A, R2 – Add contents of A and R2 and store result in A

2. ADD A, direct

Eg. ADD A, 20H - Adds contents of A and memory whose address is 20H and store result in A

3. ADD A, @Ri

Eg. ADD A, @R2 - Adds contents of A and memory whose address is given by register R2 and store result in A

4. ADD A, #data

Eg. ADD A, #20H

Adds the contents of A and 20H

5. ADDC A, Rn

Eg. ADDC A, R2 - Adds the contents of A, R2 and carry flag, and stored result in A

6. ADDC A, direct

Eg. ADDC A, 20H - Adds the contents of A, memory location whose address is 20H and the carry flag and stores result in A

7. ADDC A, @Ri

Eg. ADDC A, @R2

Adds the contents of A, memory location whose address is given by register R2 and the carry flag and stores result in the A.

8. ADDC A, #data

Eg. ADDC A, #20H

Adds the contents of A and carry flag and 20H and stores result in A

Subtraction

1. SUBB A, Rn

Eg. SUBB A, R3 - subtracts contents of R3 and carry together from A and stores results in A.

2. SUBB A, direct

Eg. SUBB A, 20H

subtracts the contents of memory location 20H and carry together from A stores result in A

3. SUBB A, @Ri

Eg. SUBB A, @R2

Subtracts the contents of memory location whose address is given by R2 and carry together from A and stores result in A

4. SUBB A, #data

Eg. SUBB A, #20H

Subtracts 20H from A and stores result in A

Multiplication and division

MUL A B.

This instruction multiplies two 8 bit unsigned numbers which are stored in A and B registers.

After multiplication the lower byte of the result will be stored in accumulator and higher byte of result will be stored in B register.

Eg. MOV A,#45H; [A]=45H
 MOV B,#0F5H; [B]=F5H
 MUL AB

Division

DIV A B.

This instruction divides the 8 bit unsigned number which is stored in A by the 8 bit unsigned number which is stored in B register.

After division the result will be stored in accumulator and remainder will be stored in B register.

Eg. MOV A,#45H
 MOV B,#0F5H
 DIV AB

DA A (Decimal Adjust After Addition).

When two BCD numbers are added, the answer is a non-BCD number. To get the result in BCD, we use DA A instruction after the addition. DA A works as follows.

If lower nibble is greater than 9 or auxiliary carry is 1, 6 is added to lower nibble.

If upper nibble is greater than 9 or carry is 1, 6 is added to upper nibble.

LOGICAL INSTRUCTIONS

ANL Destination, Source

ANL performs the bitwise logical- AND operation between the variables indicated and stores the result in the destination variable. No flags are affected.

1. ANL A, Rn

Ex. ANL A, R2 - Logically ANDs A and R2 and store result in A

2. ANL A, direct

Ex. ANL A, 20H

Logically ANDs contents of A and memory location whose address is 20H and stores result in A

3. ANL A, @Ri

Ex. ANL A,@R1 - Logically ANDs contents of A and memory location whose address is given by R1 and stores result in A

4. ANL A,#DATA

Ex. ANL A,#50H - Logically ANDs contents of A with 50H and stores result in A

5. ANL DIRECT,A

Eg. ANL 20H, A - Logically ANDs contents of A with the contents of memory location 20H and stores result at memory location 20H

6. ANL DIRECT, #DATA

Ex. ANL 20H, #30H - Logically ANDs the contents of memory location 20H with data 30H and stores result in memory location 20H

Logical OR

ORL destination, source:

ORL performs the bitwise logical – OR operation between the indicated variables, storing the results in the destinations byte. No flags are affected

1. ORL A, Rn

Ex. ORL A, R2 - Logically ORs the contents of A and R2 and stores result in A

2. ORL A, direct

Ex. ORL A, 20H

Logically ORs contents of A and memory location 20H and stores result in A

3. ORL A, @Ri

Ex. ORL A, @R2 - - Logically ORs the contents of A and memory location whose address is given by register R2 and stores result in A

4. ORL A, #data

Ex. ORL A, #32H - Logically ORs contents of A with 32H and stores result in A

5. ORL direct, A

Ex. ORL 20H, A – Logically ORs contents of A with the contents of memory location 20H and stores result at memory location 20H

6. ORL direct, #data

Ex. ORL 20H, #30H – Logically ORs the contents of memory location 20H with data 30H and stores result in memory location 20H

Logical XOR

XRL destination, source

XRL performs the bitwise logical Exclusive - OR operation between the indicated variables, storing the results in the destination. No flags are affected.

1. XRL A, Rn

Ex. XRL A, R2 - Logically XOR the contents of A and R2 and stores result in A

2. XRL A, direct

Ex. XRL A, 20H - Logically XORs the contents of A with memory location 20H and stores result in A

3. XRL A, @Ri

Ex. XRL A, @R2 - - Logically XORs the contents of A and the memory location whose address is given by register R2 and stores result in A

4. XRL A, #data

Ex. XRL A, #40H - Logically XORs the contents of A with data 40H and stores result in A

5. XRL direct, A

Ex. XRL 20H, A – Logically XORs the contents of A with the contents of memory location 20H and stores result at memory location 20H

6. XRL direct, #data

Ex. XRL 30H, #40H – Logically XORs the contents at memory location 30H with data 40H and stores result in memory location 30H

Logical NOT

CPL complements operand, leaving the result in operand. If operand is a single bit then the state of the bit will be reversed.

1. CPL A - Complement Accumulator
2. CPL C - Complement Carry

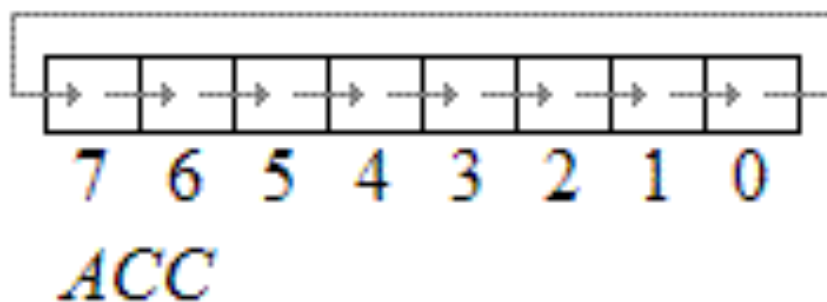
CPL Bit address

1. SWAP A – Swap the upper nibble and lower nibble of A.

Rotate Instructions

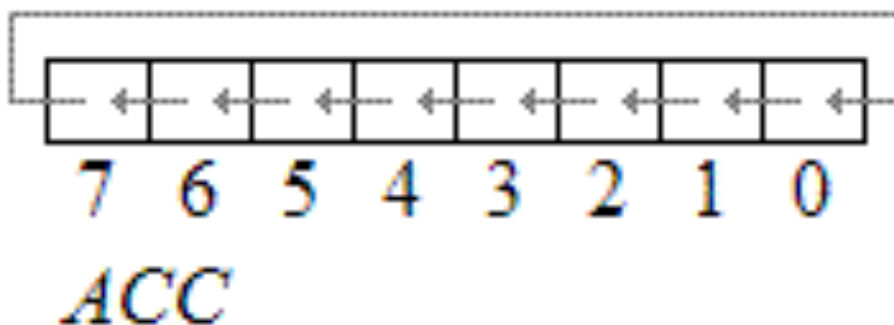
1. RR A

This instruction is rotate right the accumulator. Its operation is illustrated below. Each bit is shifted one location to the right, with bit 0 going to bit 7.



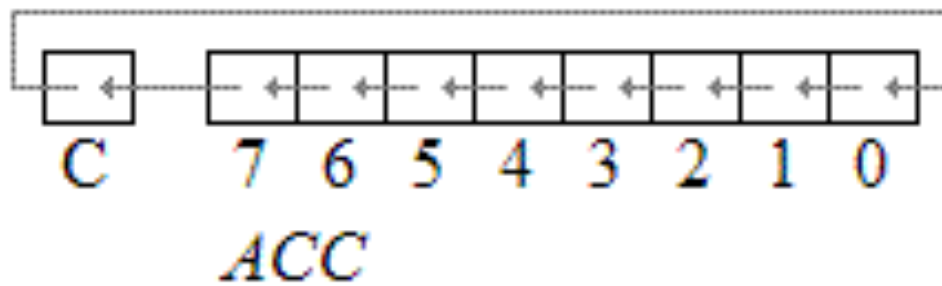
2. RL A - Rotate accumulator Left

Each bit is shifted one location to the left, with bit 7 going to bit 0.



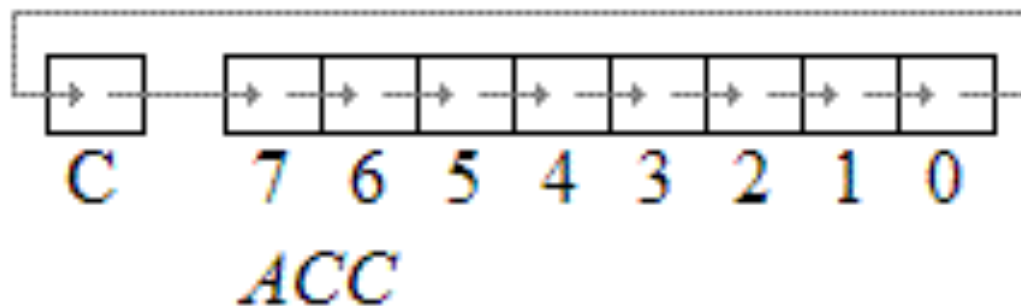
RLC A - Rotate Accumulator Left through the carry flag

The eight bits in the Accumulator and the carry flag are together rotated one bit to the left. Bit 7 moves into the carry flag; the original state of the carry flag moves into the bit 0 position. No other flags are affected.



RRC A

Rotate right through the carry. Each bit is shifted one location to the right, with bit 0 going into the carry bit in the PSW, while the carry was at goes into bit 7



Branch Instruction

- AJMP addr11 - Absolute jump
- LJMP addr16
- SJMP rel
- JMP @A+DPTR
- JZ rel
- JNZ rel
- JC rel
- JNC rel
- JB bit, rel
- JNB bit,rel

JBC bit, rel

Branch (jump) Instructions

- Branch instruction group consists of jump and call instructions.

- Jump and CALL instructions change the flow of the program by changing the contents of program counter.
- A jump permanently changes the program flow whereas call temporarily changes the program flow to allow another part of the program to run.
- There are jump instructions which change the program flow if certain condition exists.
- For example, CJNE (compare and jump if not equal). This instruction compares the magnitude of the first two operands, and changes program flow if their values are not equal.
- There are 3 types of jump instructions. They are:-

1. Relative jump (Range : (+127 to -127)Decimal, (+7FH to - 80H)
2. Short Absolute jump : (Range 0000H to 07FFH)
3. Long Absolute jump : (Range 0000H to FFFFH)

Relative Jump

Jump allows only to a range of bytes within +127 or -128(decimal) (+7FH to -80H) from the instruction following jump.

Relative jump instruction

(i) Unconditional jump

SJMP <relative address> Jump to the address specified unconditionally. Jump allows only to relative range.

(ii) Conditional jump

Bit Level jump instructions

1. JC <relative address>

Jump if CY = 1 (ie, Jump if carry is set)

If the carry flag is set branch to the address indicated; Otherwise proceed with the next instruction. No flags are affected.

2. JNC <relative address>

Jump if CY = 0 (ie., Jump if carry not set)

If the carry flag is a zero, branch to the address indicated; Otherwise proceed with the next instruction. The carry flag is not modified

3. JB bit, <relative address>

Jump if direct bit set

If the indicated bit is one, jump to the address indicated; otherwise proceed with the next instruction. The bit tested is not modified. No flags are affected.

Eg; JB 08, <relative address>

Jump to the relative address specified if the bit with address 08 = 1

4. JNB bit, <relative address >

If the indicated bit is zero, jump to the indicated address ; otherwise proceed with the next instruction. The bit tested is not modified. No flags are affected.

Eg; JNB 08, <relative address >

Jump to the relative address specified if the bit with address 08 = 0

5. JBC bit, <rel>

Jump if Bit is set and clear bit

If the indicated bit is one, branch to the address indicated; Otherwise proceed with the next instruction. The bit will not be cleared if it is already a zero.

BYTE LEVEL JUMP INSTRUCTION

1. CJNE <Destination byte>, <Source byte>, <relative address>

Compare and jump if not equal.

CJNE compares the magnitudes of the first two operands, and branches if their values are not equal. The carry flag is set if the unsigned integer values of <destination byte> is less than the unsigned integer value of <source byte>; otherwise, the carry is cleared. Neither operand is affected.

Compare the two bytes of data (destination byte and source byte) and if these are not equal then jump to the relative address specified.

For Eg. CJNE A, 56, < relative address>

Compare the data at A and data at RAM location 56 and if these are not equal then jump to the relative address specified.

2. DJNZ <byte>, <relative address>

Decrement and jump if Not zero.

DJNZ decrements the location indicated by 1, and branches to the address indicated by the second operand if the resulting value is not zero. An original value of 00H and underflow to 0FFH. No flags are affected.

Ex. DJNZ Rn, < relative address>

Decrement the data at Register Rn and jump to the relative address specified if the decremented value not equal to zero.

3. JZ <relative address>

Jump if accumulator zero.

If all bits of the accumulator are zero, branch to the address indicated; Otherwise proceed with the next instruction. The accumulator is not modified. No flags are affected.

4. JNZ rel - jump if Accumulator not zero

If any bit of the Accumulator is a one, branch to the indicated address; otherwise proceed with the next instruction. The accumulator is not modified. No flags are affected.

- There are two types of CALLS. ACALL and LCALL
- There two unconditional jumps. LJMP and SJMP

1. AJMP addr 11 Absolute jump

AJMP transfers program execution to the indicated address. Since address is 11 bit the destination must therefore be within the same 2K block of program memory as the first byte of the instruction following AJMP. No flags are affected.

2. LJMP addr 16 Long jump

Applications that need to access the entire program memory from 0000H to FFFFH use long absolute jump.

Jump allows to any locations among the 64 K memory.

LJMP <address>

Jump to the address specified unconditionally. Jump allows to anywhere in the full 64K program memory address space.

3. SJMP relative address (Short jump)

In 8051, 64 K byte of program memory space is divided into 32 pages of 2K byte each.

Jump allows only to a range of bytes within the 2K byte from the instruction following jump.

Example of short absolute jump: -

AJMP <address>

Jump to the address specified unconditionally. Jump allows only to 2K range.

CALL and Return Instruction

Subroutines are subprograms used to simplify the programs whenever a particular task is to be done repeatedly.

Subroutines are handled by CALL and RET instructions

1. ACALL addr 11 (Absolute call)

This is absolute call instruction which unconditionally calls the subroutine located at the specified address. Call the subroutine located on the same page. Push the address of next instruction immediately after the LCALL instruction on to the stack.

2. LCALL addr16 (Long call)

This is long call instruction which unconditionally calls the subroutine located at the indicated 16 bit address. Call the subroutine located anywhere in the program memory space. Push the address of next instruction immediately after the LCALL instruction on to the stack.

3. RET - Return from subroutine

RET pops the high and low – order bytes of the PC successively from the stack, decrementing the stack pointer by two program execution continues at the resulting address. No flags are affected.

4. RETI - Return from interrupt

RETI pops the high and low – order bytes of the PC successively from the stack, and restores the interrupt logic to accept additional interrupts at the same priority level as the one just processed.

The stack pointer is left decremented by two. No other registers are affected; the PSW is not automatically restored to its pre – interrupt status. Program execution continues at the resulting address, which is generally the instruction immediately after the point at which the interrupt request was detected.

If a lower – level or same – level interrupt had been interrupt is processed.

Bit Manipulation Instruction

. CLR C - Clear carry flag

- CLR bit
- SETB C
- SETB bit
- CPL C
- CPL bit
- ANL C, /bit
- ANL C, bit
- ORL C, /bit
- MOV C, bit

MOV bit, C

Bit manipulation Instruction

- 8051 has 128 bit addressable memory, bit addressable SFRs and bit addressable PORT pins.

- It is possible to perform following bit wise operations for these bit addressable locations.

1. CLR C - Clear carry flag

2. CLR bit - clear direct bit

The indicated bit is cleared (reset to zero). No other flags are affected. CLR can operate on the carry flag or any directly addressable bit.

3. SETB C – Set direct bit

SETB sets the indicated bit to one. SETB can operate on the carry flag or any directly addressable bit. No other flags are affected.

4. SETB bit - Set bit

SETB sets the indicated bit to one. SETB can operate on the carry flag or any directly addressable bit. No other flags are affected.

5. CPL bit – Complement direct bit

The bit variable specified is complemented. No other flags are affected.

6. CPL C – Complement carry flag

7. ANL C, bit - AND direct bit to carry flag

Logically AND carry and content of bit address, store result in carry.

8. ANL C, /Bit - AND complement of direct bit to carry

9. ORL C, bit - OR direct bit to Carry flag (C= C OR bit)

Logically OR carry and content of bit address, store result in carry.

10. ORL C, /bit – OR complement of direct bit to carry

11. MOV C, bit - Move direct bit to Carry flag

12. MOV bit, C – Move carry flag to direct bit

Data types and Directives

Data types

- Microcontroller 8051 has only one 8-bits data type and the size of each register is also 8 bits.
- The job of the programmer is to break down data larger than 8 bits [00 to FFH, or 0 to 255 in decimal] to be processed by the CPU.
- The data type used by the 8051 can be positive or negative.

- The most significant bit of 8 – bit data indicates the sign of the data.
- If the most significant bit is zero, the number is positive; otherwise, the number is negative.

Assembler Directives

Directives or assembler directives are instructions given to the assembler. The following are directives used in 8051

DB (Define Byte)

- The DB directive is used to define the 8-bit data and the numbers can be in decimal, binary, hex or ASCII formats.
- For decimal, the "D" after the decimal number is optional, but using "B" [binary] and "H" [hexadecimal] is required. The assembler will convert the numbers in hex.
- To indicate ASCII, simply put it in quotation marks 'like this'. The assembler will assign the ASCII code for the numbers or characters automatically.
- The DB directive is the only directive that can be used to define ASCII strings larger than two characters. It should be used for all ASCII data definitions.
- For Example,

ORG 300H

DATA1: DB 30 ;Decimal number [1E in HEX]

DATA2: DB 01010101 Binary ;Binary [55 in HEX]

DATA3: DB 32H ; equal to 32 in Hex, ie., 50 in decimal

DATA4: DB "A" ; equal to ASCII A

DATA5: DB "259" ; ASCII Character

DATA6: DB "MY NAME IS AYESHA" ; ASCII Character

ORG [Origin]:

- The ORG directive is used to indicate the beginning of the address.
- The number that comes after ORG can be either in HEX or in decimal.
- If the number is not followed by 'H', it is decimal and the assembler will convert it into hex.

For Example ORG 0100H

- Some assemblers use ".ORG" [notice the dot] instead of "ORG" for the origin directive. For that you need to check your assembler.

EQU [Equate]:

- EQU is used to define a constant without occupying a memory location.
- Here uses EQU for the counter constant and then the constant is used to load the R4 register.
- Example

```
COUNT EQU 22
```

```
....    ....
```

```
MOV R4,#COUNT
```

After executing the instruction "MOV R4,#COUNT", the register R4 will be loaded with the value 22 [# sign indicates it is a value, notice it].

END Directive:

- This indicates to the assembler the end of the source (asm)file
- END is the last line of 8051 program
- After the END, anything written will be ignored by the assembler

Time Delay Programmes

Machine cycle for the 8051

- The CPU takes a certain number of clock cycles to execute an instruction. In the 8051 family, these clock cycles are referred to as machine cycles.
- In the 8051 family, the length of the machine cycle depends on the frequency of the crystal oscillator connected to the 8051 system.
- The crystal oscillator, along with on-chip circuitry, provide the clock source for the 8051 CPU
- The frequency of the crystal connected to the 8051 family can vary from 4MHZ to 30MHZ, depending on the chip rating and manufacturer.
- Very often the 11.0592 MHZ crystal oscillator is used to make the 8051 based system compatible with the serial port of the IBM PC.
- In the original 8051, one machine cycle lasts 12 oscillator periods.
- **Therefore to calculate the machine cycle for the 8051, we take 1/12 of the crystal frequency, then take its inverse.**

Example

1. The following crystal frequency for three different 8051 based systems. Find the period of the machine cycle in each case.

(a) 11.0592 MHz (b) 16MHz (c) 20MHz

Solution

(a) 11.0592MHz

$$\frac{1}{12} * 11.0592 \text{MHz} = 921.6 \text{kHz},$$

$$\text{Machine cycle is } \frac{1}{921.6 \text{kHz}} = 1.085 \mu\text{s}$$

(b) 16 MHz

$$\frac{1}{12} * 16 \text{MHz} = 1.333 \text{MHz},$$

$$\text{Machine cycle is } \frac{1}{1.333 \text{MHz}} = 0.75 \mu\text{s}$$

(c) 20 MHz

$$\frac{1}{12} * 20 \text{MHz} = 1.66 \text{MHz},$$

$$\text{Machine cycle is } \frac{1}{1.66 \text{MHz}} = 0.60 \mu\text{s}$$

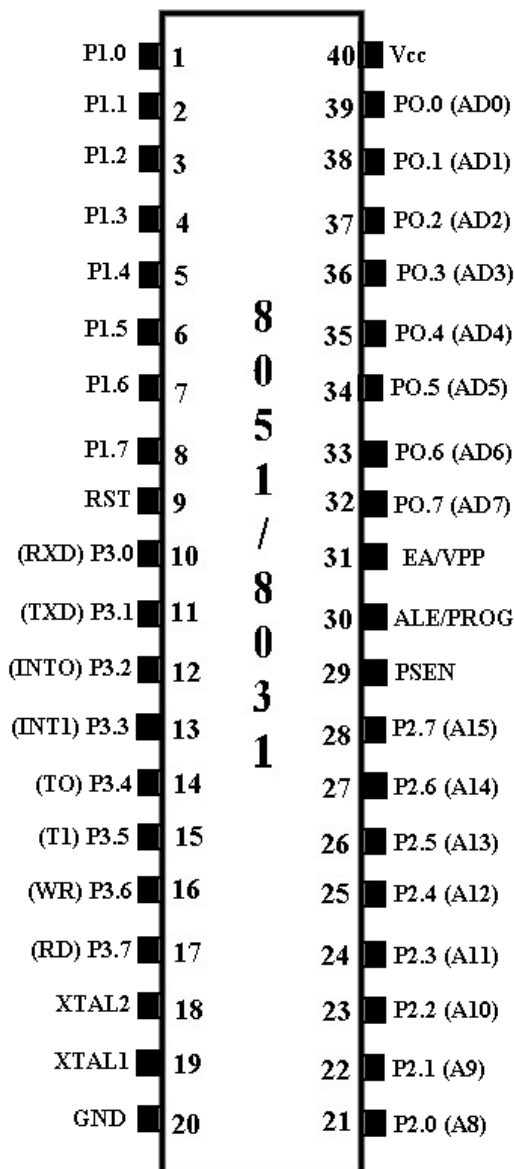
I/O port programming

- In the 8051 there are a total of four ports for I/O operations, 40 pins, a total of 32 pins are set aside for the four ports P0, P1, P2 and P3, where each port takes 8 pins. The rest of the pins are designated as Vcc, GND, XTAL1, XTAL2, RST, EA, ALE/PROG and PSEN

I/O port pins and their functions

- There are four ports P0, P1, P2 and P3 each use 8 pins, making them 8-bit ports. All the ports upon RESET are configured as inputs, ready to be used as output ports. To use any of these ports as an input port, it must be programmed.

Pin configuration of 8051 microcontroller



Port 0

- Port 0 occupies a total of 8 pins (pins 32-39) .It can be used for input or output.
- To use the pins of port 0 as both input and output ports, each pin must be connected externally to a 10K ohm pull-up resistor. This is due to the fact that P0 is an open drain, unlike P1, P2, and P3.
- Open drain is a term used for MOS chips in the same way that open collector is used for TTL chips.
- With external pull-up resistors connected upon reset, port 0 is configured as an output port. For example, the following code will continuously send out to port 0 the alternating values 55H and AAH

```

MOV A, #55H
BACK:  MOV P0, A

```

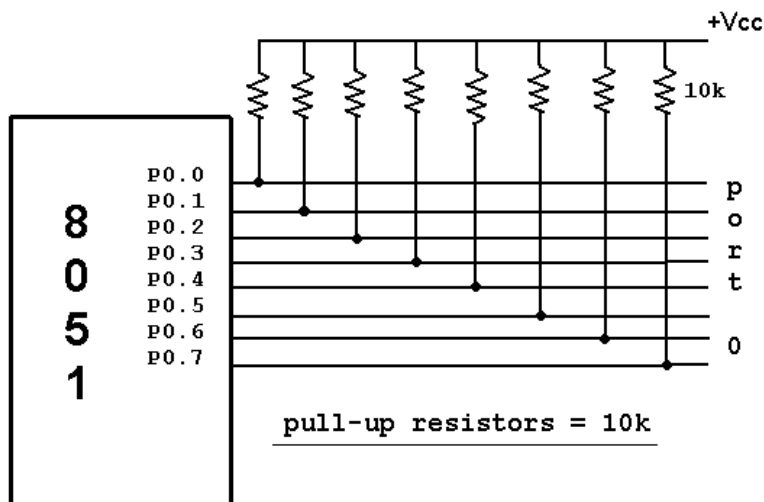
```

ACALL DELAY
CPL A
SJMP  BACK

```

Port 0 as Input

- With resistors connected to port 0, in order to make it an input, the port must be programmed by writing 1 to all the bits.
- In the following code, port 0 is configured first as an input port by writing 1's to it, and then data is received from the port and sent to P1.



```

MOV A, #0FFH      ;A=FF hex
MOV P0, A         ; make p0 an input port
                  ; by writing all 1s to it
BACK:  MOV A,P0    ; get data from P0
        MOV P1, A  ; send it to port 1
        SJMP  BACK ; keep doing it

```

Dual role of port 0:

Port 0 is also designated as AD0-AD7, allowing it to be used for both address and data. When connecting an 8051/31 to an external memory, port 0 provides both address and data.

The 8051 multiplexes address and data through port 0 to save pins. ALE indicates if P0 has address or data.

When ALE = 0, it provides data D0-D7, but when ALE = 1 it has address and data with the help of a 74LS373 latch.

Port 1

- Port 1 occupies a total of 8 pins (pins 1 through 8). It can be used as input or output.
- In contrast to port 0, this port does not need any pull-up resistors since it already has pull-up resistors internally. Upon reset, Port 1 is configured as an output port.

For example, the following code will continuously send out to port1 the alternating values 55H & AAH

```
        ; toggle all bits of P1 continuously
        MOV A,#55H           ; A = 55 hex
BACK:    MOV P1,A             ; send it to Port 1
        ACALL DELAY          ; call delay routine
        CPL A                ; Complement register A
        SJMP BACK
```

Port 1 as input

- If port 1 has been configured as an output port, to make it an input port again, it must be programmed as such by writing 1 to all its bits. In the following code port1 is configured first as an input port by writing 1's to it, then data is received from the port and saved in R7, R6 & R5.

```
        MOV A,#0FFH          ; A=FF HEX
        MOV P1,A             ; make P1 an input port by writing all 1's to it
        MOV A,P1             ; get data from P1
        MOV R7,A             ; save it in register R7
        ACALL DELAY          ; wait
        MOV A,P1             ; get another data from P1
        MOV R6,A             ; save it in register R6
        ACALL DELAY          ; wait
        MOV A,P1             ; get another data from P1
        MOV R5,A             ; save it in register R5
```

Port 2

- Port 2 occupies a total of 8 pins (pins 21- 28).
- It can be used as input or output. Just like P1, P2 does not need any pull-up resistors since it already has pull-up resistors internally. Upon reset, Port 2 is configured as an output port.

For example, the following code will send out continuously to port 2 the alternating values 55h and AAH. That is all the bits of port 2 toggle continuously

```

MOV A,#55H          ; A = 55 hex
BACK: MOV P2,A       ;send it to Port 2
ACALL DELAY         ;call delay routine
CPL A               ;make A=0
SJMP BACK

```

Port 2 as input

- To make port 2 an input, it must be programmed as such by writing 1 to all its bits.
- In the following code, port 2 is configured first as an input port by writing 1's to it.
- Then data is received from that port and is sent to P1 continuously.

```

          MOV A,#0FFH    ;A=FF hex
          MOV P2,A        ;make P2 an input port by writing all 1's to it
BACK:     MOV A,P2        ;get data from P2
          MOV P1,A        ;send it to Port1
          SJMP BACK       ;keep doing that

```

Dual role of port 2

- In systems based on the 8751, 8951, and DS5000, P2 is used as simple I/O. However, in 8031-based systems, port 2 must be used along with P0 to provide the 16-bit address for the external memory.
- As shown in pin configuration 8051, port 2 is also designed as A8-A15, indicating the dual function. Since an 8031 is capable of accessing 64K bytes of external memory, it needs a path for the 16 bits of the address. While P0 provides the lower 8 bits via A0-A7, it is the job of P2 to provide bits A8-A15 of the address.
- In other words, when 8031 is connected to external memory, P2 is used for the upper 8 bits of the 16 bit address, and it cannot be used for I/O.

Port 3

- **Port 3 :** Port 3 occupies a total of 8 pins, pins 10 through 17. It can be used as input or output.
- P3 does not need any pull-up resistors, the same as P1 and P2 did not.
- Although port 3 is configured as an output port upon reset.
- Port 3 has the additional function of providing some extremely important signals such as interrupts.
- This information applies both 8051 and 8031 chips.

PORT 3 ALTERNATE FUNCTIONS :

P3 BIT	FUNCTION	PIN
P3.0	RXD	10
P3.1	TXD	11
P3.2	$\overline{\text{INT0}}$	12
P3.3	$\overline{\text{INT1}}$	13
P3.4	TO	14
P3.5	TI	15
P3.6	$\overline{\text{WR}}$	16
P3.7	$\overline{\text{RD}}$	17

- P3.0 and P3.1 are used for the RxD and TxD serial communications signals.
- Bits P3.2 and P3.3 are set aside for external interrupts. Bits P3.4 and P3.5 are used for timers 0 and 1.
- Finally P3.6 and P3.7 are used to provide the WR and RD signals of external memories connected in 8031 based systems.

I/O Bit Manipulation programming

- Single bit addressability of ports: There are times that we need to access only 1 or 2 bits of the port instead of the entire 8 bits. A powerful feature of 8051 I/O ports is their capability to access individual bits of the port without altering the rest of the bits in that port.
- For example, the following code toggles the bit p1.2 continuously.

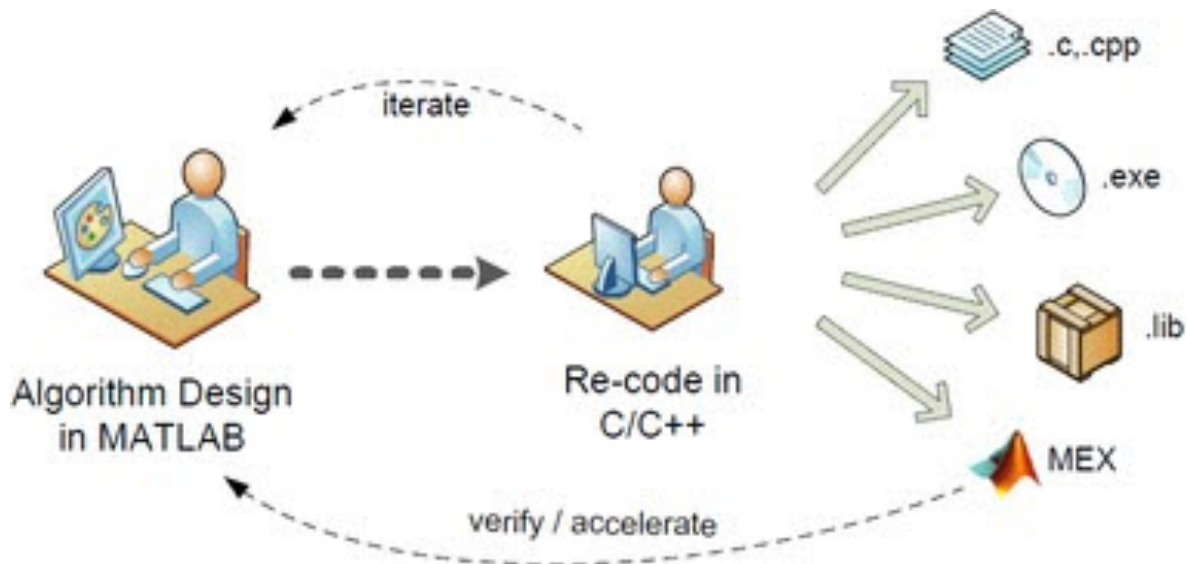
```
BACK: CPL P1.2      ;complement p1.2 only
      ACALL DELAY
      SJMP BACK
```

SINGLE-BIT ADDRESSABILITY OF THE PORTS :

P0	P1	P2	P3	PORT BIT
p0.0	p1.0	p2.0	p3.0	D0
p0.1	p1.1	p2.1	p3.1	D1
p0.2	p1.2	p2.2	p3.2	D2
p0.3	p1.3	p2.3	p3.3	D3
p0.4	p1.4	p2.4	p3.4	D4
p0.5	p1.5	p2.5	p3.5	D5
p0.6	p1.6	p2.6	p3.6	D6
p0.7	p1.7	p2.7	p3.7	D7

Embedded programming in C

- Embedded C Programming is the soul of the processor functioning inside each and every embedded system we come across in our daily life, such as mobile phone, washing machine, and digital camera.
- Each processor is associated with an embedded software. The first and foremost thing is the embedded software that decides functioning of the embedded system. Embedded C language is most frequently used to program microcontroller.



- Earlier, many embedded applications were developed using assembly level programming.
- However, they did not provide portability. This disadvantage was overcome by the advent of various high level languages like C, Pascal, and COBOL.
- However, it was the C language that got extensive acceptance for embedded systems, and it continues to do so.
- The C code written is more reliable, scalable, and portable; and in fact, much easier to understand.

Differences between C and Embedded C

C programming	Embedded C programming
Possesses native development in nature.	Possesses cross development in nature.
Independent of hardware architecture.	Dependent on hardware architecture (microcontroller or other devices).
Used for Desktop applications, OS and PC memories.	Used for limited resources like RAM, ROM and I/O peripherals on embedded controller.

Why program the 8051 in C

- Compilers produce hex files that is downloaded to ROM of microcontroller
- The size of hex file produced by the compiler is one of the main concerns of microcontroller programmers, for two reasons
 1. Microcontrollers have limited on-chip ROM
 2. The code space for the 8051 is limited to 64K bytes.

C programming is less time consuming, but has larger hex file size

The reasons for writing programs in C

- It is easier and less time consuming to write in C than Assembly
- C is easier to modify and update
- You can use code available in function libraries
- C code is portable to other microcontroller with little of no modification

Data types and Time delay in 8051C

Data types

A good understanding of C data types for 8051 can help programmers to create smaller hex files

- Unsigned char
- Signed char
- Unsigned int
- Signed int
- Sbit (single bit)
- Bit and sfr

Unsigned char

- Since the 8051 is an 8 bit microcontroller, the character data type is the most natural choice for many applications.
- The unsigned char is an 8-bit data type that takes a value in the range of 0 – 255 (00 – FFH).
- In situations where + and – are needed to represent a given quantity such as temperature, the use of the signed char data is a must.

- unsigned char z;

Example (unsigned)

Example 7-1 (unsigned char)

Write an 8051 C program to send values 00-FF to port P1.

Solution:

```
#include <reg51.h>
void main(void)
{
    unsigned char z;
    for (z=0; z<=255; z++)
        P1=z;
}
```

Example 7-2 (unsigned char)

Write an 8051 C program to send hex values for ASCII characters of 0,1,2,3,4,5,A,B,C, and D to port P1.

Solution:

```
#include <reg51.h>
void main(void) {
    unsigned char mynum[]="012345ABCD";
    unsigned char z;
    for (z=0; z<10; z++)
        P1=mynum[z];
}
```

Note: "012345ABCD" is stored in RAM and can be changed.

Example 7-3 (unsigned char)

Write an 8051 C program to toggle all the bits of P1 continuously.

Solution:

```
#include <reg51.h>
void main(void) {
    for ( ; ; ) {    //repeat forever
        P1=0x55;      //0x: in hex (binary)
        P1=0xAA;
    }
}
```

Signed char

- The signed char is an 8 bit data type that uses the most significant bit (D7 of D7 – D0) to represent the – or + value.
- As a result, we have only 7 bits for the magnitude of the signed number, giving us values from -128 to +127.

Example 7-4 (signed char)

Write an 8051 C program to send values of -4 to 4 to port P1.

Solution:

```
#include <reg51.h>
void main(void)
{
    char mynum[]={+1,-1,+2,-2,+3,-3,+4,-4};
    unsigned char z;
    for (z=0; z<8; z++)
        P1=mynum[z];
}
```

Unsigned int

- The unsigned int is a 16-bit data type that takes a value in the range of 0 to 65535 (0000 – FFFFH)
- In the 8051, unsigned int is used to define 16-bit variables such as memory addresses .
- It is also used to set counter values of more than 256.

- Since the 8051 is an 8 bit microcontroller and the int data type takes two bytes of RAM, we must not use the int data type unless we have to.
- Since registers and memory accesses are in 8-bit chunks, the misuse of int variables will result in a larger hex file
- Such misuse is not a big deal in PCs with 256 megabytes of memory, 32 bit Pentium registers and memory accesses, and a bus speed of 133MHz.

Example 7-5 (unsigned int, sbit)

Write an 8051 C program to toggle bit D0 of P1 50,000 times.

Solution:

```
#include <reg51.h>
sbit MYBIT=P1^0;
void main(void) {
    unsigned int z;
    for (z=0;z<50000;z++) {
        MYBIT=0;
        MYBIT=1;
    }
}
```

Signed int

- Signed int is a 16 bit data type that uses the most significant bit (D15 – D0) to represent the – or + value.
- As a result, we have only 15 bits for the magnitude of the number, or values from -32,768 to +32,767

Sbit (Single bit)

- The sbit keyword is a widely used 8051 C data type designed specifically to access single – bit addressable registers.
- It allows access to the single bits of the SFR registers.
- Some of the SFRs are bit – addressable.
- Among the SFRs that are widely used and are also bit, addressable are ports P0 – P3.
- Eg: sbit MYBIT = P1^0

Bit and sfr

- The bit data type allows access to single bits of bit – addressable memory spaces 20 – 2FH.
- Notice that while the sbit data type is used for bit – addressable SFRs, the bit data type used for the bit – addressable section of RAM space 20 – 2FH.
- To access the byte-size SFR registers, use the *sfr* data type of RAM addresses space 80 – FFH
- Eg: sfr mydata = P1

Data Type	Size in Bits	Data Range/Usage	
unsigned char	8-bit	0 to 255	
(signed) char	8-bit	-128 to +127	
unsigned int	16-bit	0 to 65535	
(signed) int	16-bit	-32768 to +32767	
sbit	1-bit	SFR bit-addressable	only
bit	1-bit	RAM bit-addressable	only
sfr	8-bit	RAM addresses 80 –	FFH only

Time Delay

- There are two ways to create a time delay in 8051 C
 1. Using a simple for loop
 2. Using the 8051 timers

Table 7-1: Some Widely Used Data Types for 8051 C

Data Type	Size in Bits	Data Range/Usage
unsigned char	8-bit	0 to 255
signed char	8-bit	-128 to +127
unsigned int	16-bit	0 to 65535
(signed) int	16-bit	-32,768 to +32,767
sbit	1-bit	SFR bit-addressable only
bit	1-bit	RAM bit-addressable only
sfr	8-bit	RAM addresses 80 - FFH only

- In creating a time delay using a for loop, we must be mindful of three factors that can affect the accuracy of the delay.

1. The 8051 design

- Since the original 8051 was designed in 1980, both the fields of IC technology and microprocessor architectural design have seen great advancements.
- The number of machine cycles and the number of clock periods per machine cycle vary among different versions of the 8051 microcontroller.
- While the original 8051 design used 12 clock periods per machine cycle, many of the newer generations of the 8051 use fewer clocks per machine cycle.
- For example, the DS5000 uses 4 clock periods per machine cycle, while the DS89C420 uses only one clock per machine cycle.

2. The crystal frequency to the X1 – X2 input pins. The duration of the clock period for the machine cycle is a function of this crystal frequency.

3. Compiler choice

- The third factor that affects the time delay is the compiler used to compile the C program.
- When we program in Assembly language, we can control the exact instructions and their sequences used in the delay subroutine.
- In the case of C programs, it is the C compiler that converts the C statements and functions to Assembly language instructions.
- As a result, different compilers produce different code. In other words, if we compile a given 8051 C programs with different compilers, each compiler produces different hex code,

Write an 8051 C program to toggle bits of P1 continuously forever with some delay.

Solution:

```
#include <reg51.h>
void main(void)
{
    unsigned int x;
    for (;;)                //repeat forever
    {
        P1=0x55;
        for (x=0;x<40000;x++); //delay size//unknown
        P1=0xAA;
        for (x=0;x<40000;x++);
    }
}
```

EXAMPLE

Write an 8051 C program to toggle the bits of P1 ports continuously with a 250ms delay

```
#include <reg51.h>
Void MSDelay (unsigned int);
Void main(void)
{
    while (1)
    {
        P1 = 0*55;
        MSDelay (250);
        P1 = 0*AA;
        MSDelay (250);
    }
}
void MSDelay (unsigned bit itime)
{
    unsigned int i, j;
    for (i=0; i<itime;i++)
    for (j=0; j<1275; j++;
```

```
}
```

EXAMPLE

Write an 8051 C program to toggle the bits of P0 and P2 continuously with a 250ms delay

```
#include <reg51.h>
Void MSDelay (unsigned int);
Void main(void)
{
    while (1)
    {
        P0 = 0*55;
        P2 =0*55;
        MSDelay (250);
        P0 = 0*AA;
        P2 = 0*AA;
        MSDelay (250);
    }
}
void MSDelay (unsigned bit itime)

{

    unsigned int i, j;
    for (i=0; i<itime;i++
    for (j=0; j<1275; j++;
}
```

I/O PORT PROGRAMMING IN EMBEDDED C

Byte size I/O

- Ports P0 – P3 are byte – accessible.
- We use the P0 – P3 labels are defined in the 8051 C header file.

Example

- LEDs are connected to bits P1 and P2. Write an 8051 C program that shows the count from 0 to FFH(0000 0000 to 1111 1111 in binary) on the LEDS
- **solution**

```
#include <reg51.h>
#define LED p2      // notice how we can define p2
Void main (void)
{
    P1=00;          //clear p1
    LED = 0;         // clear p2
    for(;;)          //repeat forever
    {
        p1++;        //increment p1
        LED++;        //increments p2
    }
}
```

Example 9: Write an 8051 C program to get a byte of data form P1, wait ½ second, and then send it to P2.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
{
    unsigned char mybyte;
    P1=0xFF; //make P1 input port
    while (1)
    {
        mybyte=P1; //get a byte from P1
        MSDelay(500);
        P2=mybyte; //send it to P2
    }
}
```

```
void MSDelay (unsigned int itime)
```

```
{
    unsigned int i, j;
    for (i=0; i<itime;i++)
        for (j=0; j<1275; j++)
            ;
}
```

Example 10: Write an 8051 C program to get a byte of data from P0. If it is less than 100, send it to P1; otherwise, send it to P2.

Solution:

```
#include <reg51.h>
void main(void)
{
    unsigned char mybyte;
    P0=0xFF;           //make P0 input port
    while (1)
    { mybyte=P0;        //get a byte from P0
      if (mybyte<100)
        P1=mybyte;     //send it to P1
      else
        P2=mybyte;     //send it to P2
    }
}
```

Bit addressable I/O programming

- The I/O ports of P0 – P3 are bit – addressable. We can access a single bit without disturbing the rest of the port.
- We use the sbit data type to access a single bit of P0 –P3.
- One way to do that is to use the Px^y format where x is the port 0, 1 2, or 3, and y is the bit 0 – 7 of that port.
- For example, P1⁷ indicates P1.7.
- When using this method, you need to include the reg51.h file.

Example 11: Write an 8051 C program to toggle only bit P2.4 continuously without disturbing the rest of the bits of P2.

Solution:

```
//Toggling an individual bit
#include <reg51.h>
sbit mybit=P2^4;
void main(void)
{
    while (1)
    {
        mybit=1;      //turn on P2.4
        mybit=0;      //turn off P2.4
    }
}
```

Ports P0 – P3 are bit addressable and we use *sbit* data type to access a single bit of P0 - P3

Use the Px^y format, where x is the port 0, 2, or 3 and y is the bit 0 – 7 of that port

Example 12: Write an 8051 C program to monitor bit P1.5. If it is high, send 55H to P0; otherwise, send AAH to P2.

Solution:

```
#include <reg51.h>
sbit mybit=P1^5;
void main(void)
{
    mybit=1;           //make mybit an input
    while (1)
    {
        if (mybit==1)
            P0=0x55;
        else
            P2=0xAA;
    }
}
```

EXAMPLE

- A door sensor is connected to the P1.1 pin, and a buzzer is connected to P1.7. Write an 8051 C program to monitor the door sensor, and when it opens, sound the buzzer. You can sound the buzzer by sending a square wave of a few hundred Hz.

- **Solution:**

```
#include <reg51.h>
void MSDelay(unsigned int);
sbit Dsensor=P1^1;
sbit Buzzer=P1^7;
void main(void)
{
    Dsensor=1;           //make P1.1 an input
    while (Dsensor==1)   //while it opens
    {
        Buzzer=0;
        MSDelay(200);
        Buzzer=1;
        MSDelay(200);
    }
}

void MSDelay (unsigned int itime)

{

    unsigned int i, j;
    for (i=0; i<itime;i++
    for (j=0; j<1275; j++;
```

}

EXAMPLE

The data pins of an LCD are connected to P1. The information is latched into the LCD whenever its Enable pin goes from high to low. Write an 8051 C program “The earth is but One Country” to this LCD

```
#include <reg51.h>
#define LCDDData P1 //LCDDData declaration
sbit En=P2^0; //the enable pin
void main(void)
{
    unsigned char message[] = "The Earth is but one country" ;
    unsigned char z;
    for (z=0;z<10;z++) //send 10 characters
    {
        LCDDData=message[z];
        En=1; //a high-
        En=0; //-to-low pulse to latch data
    }
}
```

Accessing SFR addresses 80 – FFH

- Another way to access the SFR ram space 80 – FFH is to use the sfr data type.
- We can also access a single bit of any SFR if we specify the bit address.
- This allows us to access any byte of the SFR RAM space 80 – FFH.

Example 14: Write an 8051 C program to toggle all the bits of P0 and P2 continuously with a 250 ms delay. Use the sfr keyword to declare the port addresses.

Solution:

//Accessing Ports as SFRs using sfr data type

```
sfr P0=0x80;
sfr P1=0x90;
sfr P2=0xA0;
void MSDelay(unsigned int);
void main(void)
{
    while (1)
    {
        P0=0x55;
        P2=0x55;
        MSDelay(250);
        P0=0xAA;
        P2=0xAA;
        MSDelay(250);
    }
}
```

Another way to access the SFR RAM space 80 – FFH is to use the *sfr* data type

void MSDelay (unsigned int itime)

```
{
    unsigned int i, j;
    for (i=0; i<itime; i++)
        for (j=0; j<1275; j++)
            ;
}
```

Example 15: Write an 8051 C program to turn bit P1.5 on and off 50,000 times.

Solution:

sbit MYBIT=0x95;

We can access a single bit of any SFR if we specify the bit address

```
void main(void)
{
    unsigned int z;
    for (z=0; z<50000; z++)
    {
        MYBIT=1;
        MYBIT=0;
    }
}
```

Using bit data type for bit – addressable RAM

- The sbit data type is used for bit – addressable SFR registers only. Sometimes we need to store some data in a bit addressable section of the data RAM space 20 – 2FH

Example 16: Write an 8051 C program to get the status of bit P1.0, save it, and send it to P2.7 continuously.

Solution:

```
#include <reg51.h>
sbit inbit=P1^0;
sbit outbit=P2^7;
bit membit;      //use bit to declare
                  //bit- addressable memory

void main(void)
{
    while (1)
    {
        membit=inbit;      //get a bit from P1.0
        outbit=membit;      //send it to P2.7
    }
}
```

We use bit data type to access data in a bit-addressable section of the data RAM space 20 – 2FH